Raster Images

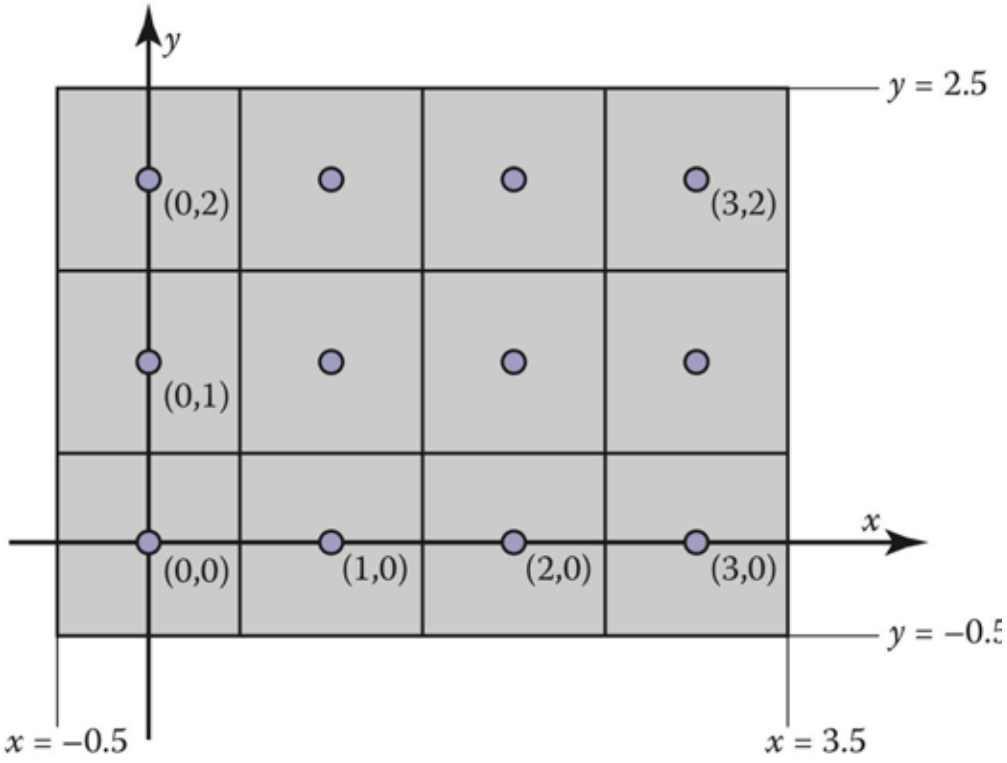
Read Chapter 3 of *Fundamentals of Computer Graphics*

What is an Image?

Image = distribution of light energy on 2D “film”
Digital images represented as rectangular arrays of **pixels**

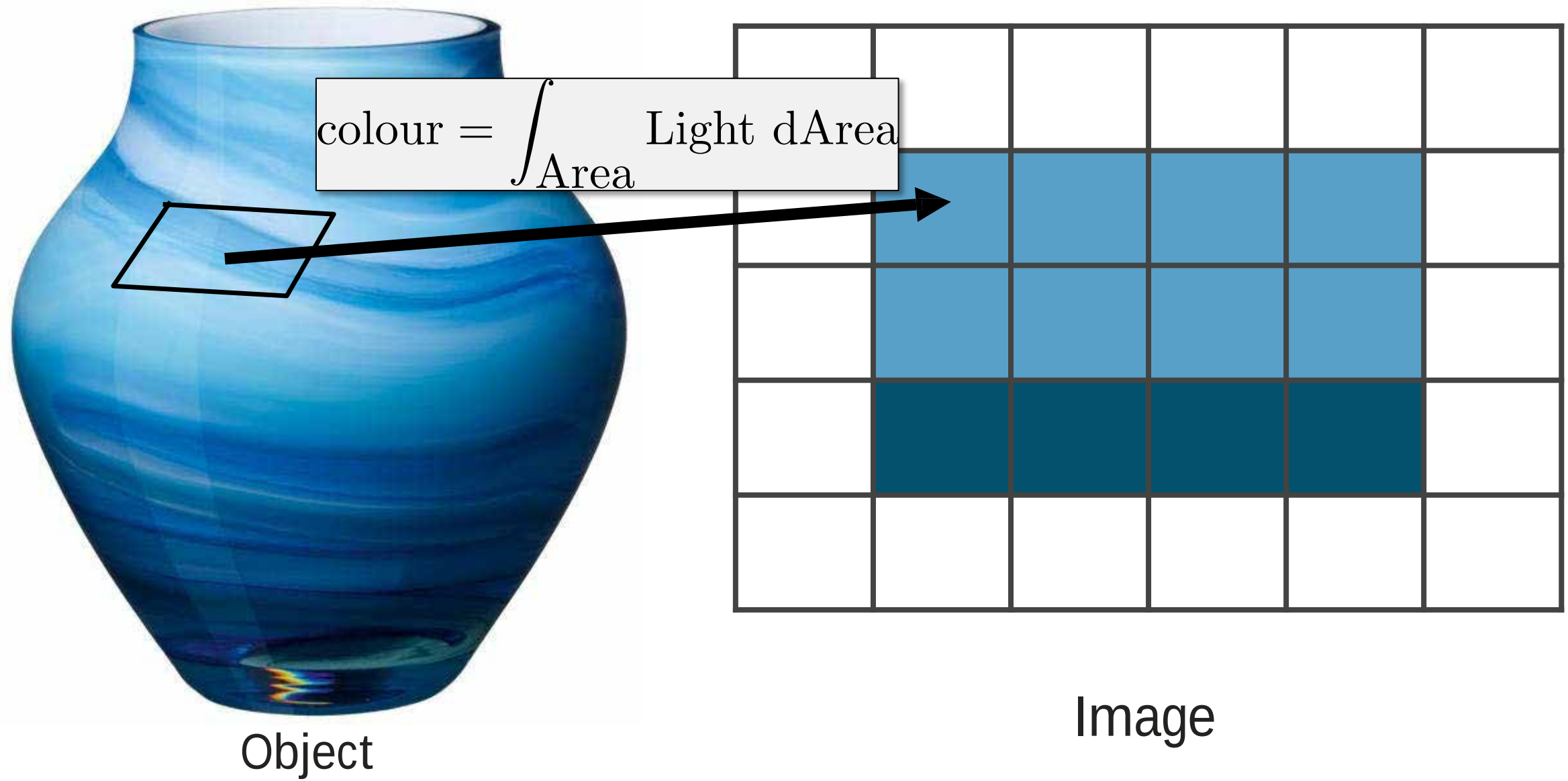
Image = a function defined for an array of pixels

$$I(x, y) : \mathbb{R}^2 \rightarrow \mathbb{R}^{+n}$$



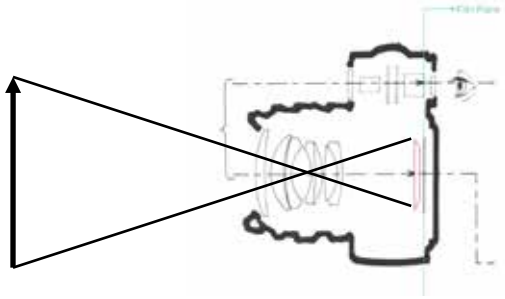
Pixel Co-ordinates

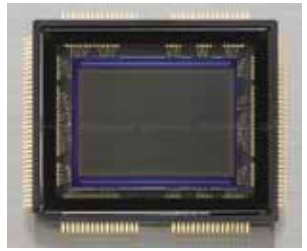
A Pixel in 3D need not be 'square'



Raster Devices and pixels

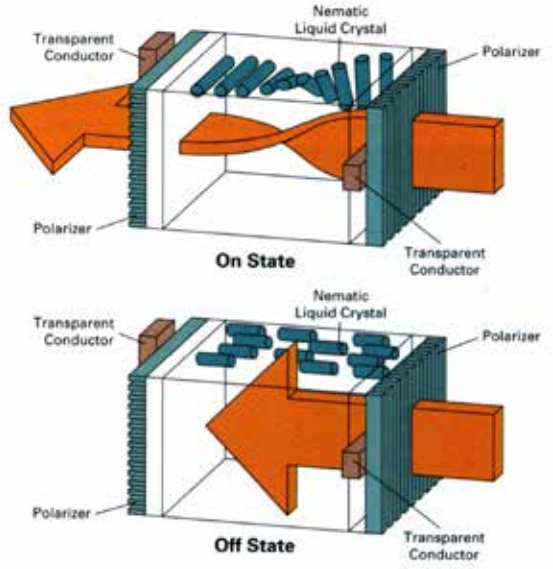
- Input (scanners, cameras)
  2D array sensor: digital camera





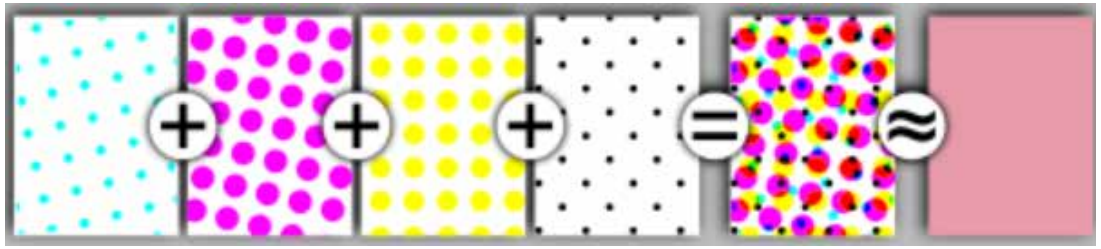
- Output (printers, displays)
  Emissive: light-emitting diode (LED)

  Transmissive: liquid crystal display (LCD)



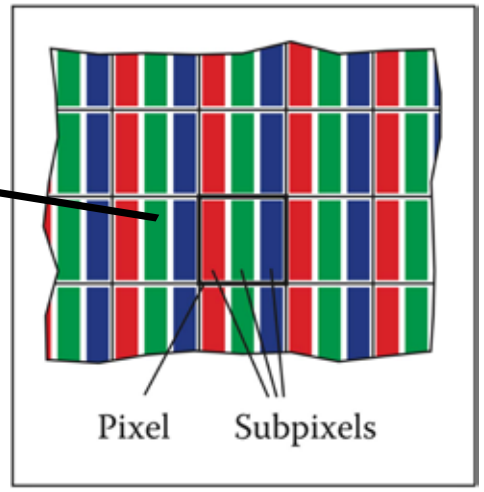
Loosely, LED screens are newer, better LCD screens!

Ink-jet printer Absorptive:
Cyan ink absorbs red, Magenta absorbs green and Yellow absorbs blue light.

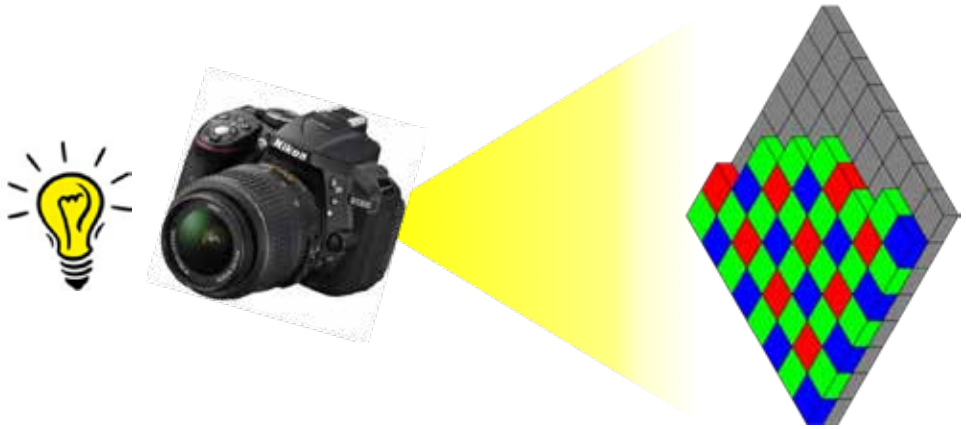


Raster Input/Output

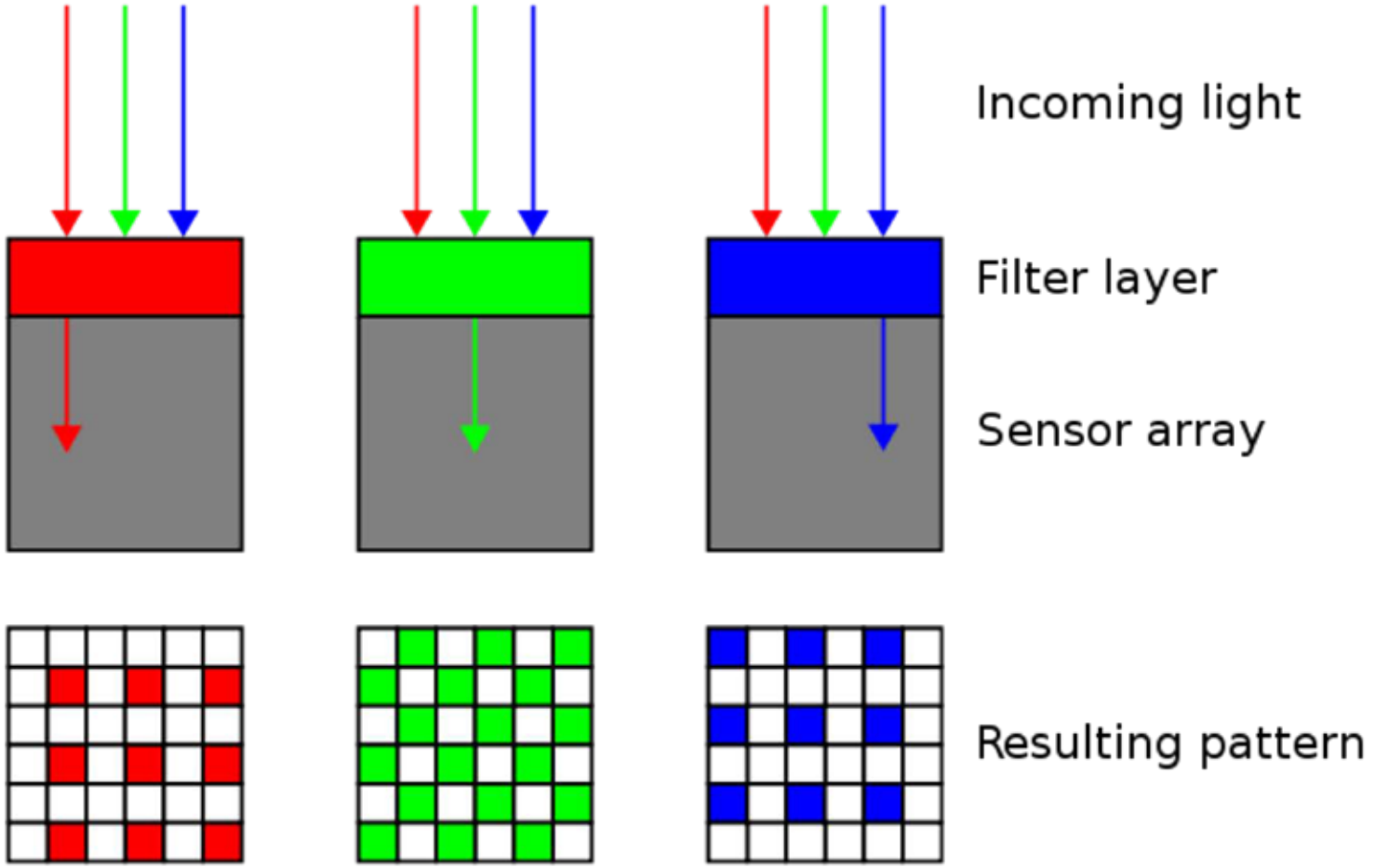
Bayer Filter



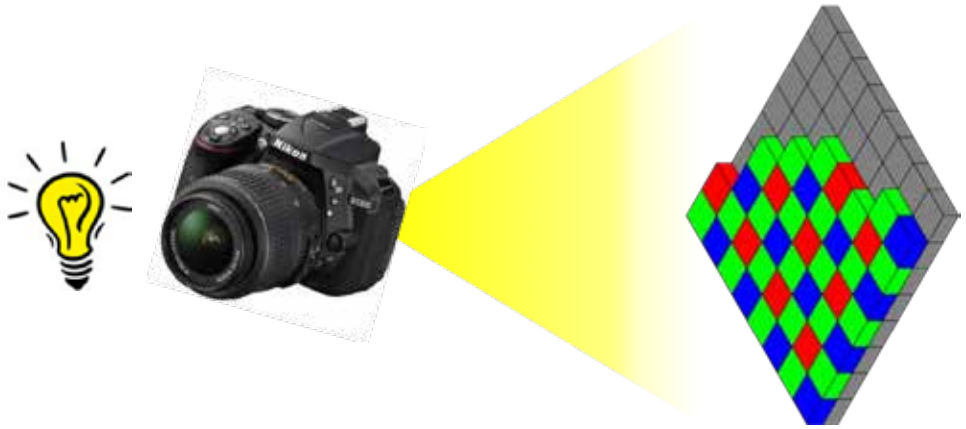
Raster Input Devices



$$i = 0.2126r + 0.7152g + 0.0722b.$$

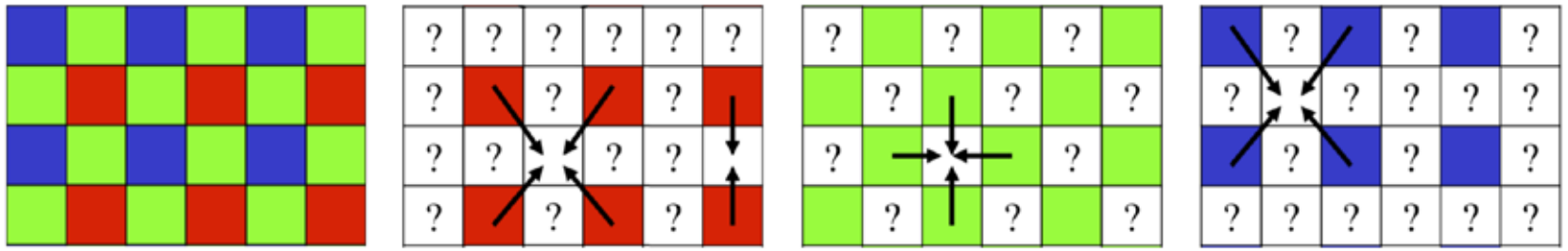


Why more ‘green’ than ‘red’ or ‘blue’?

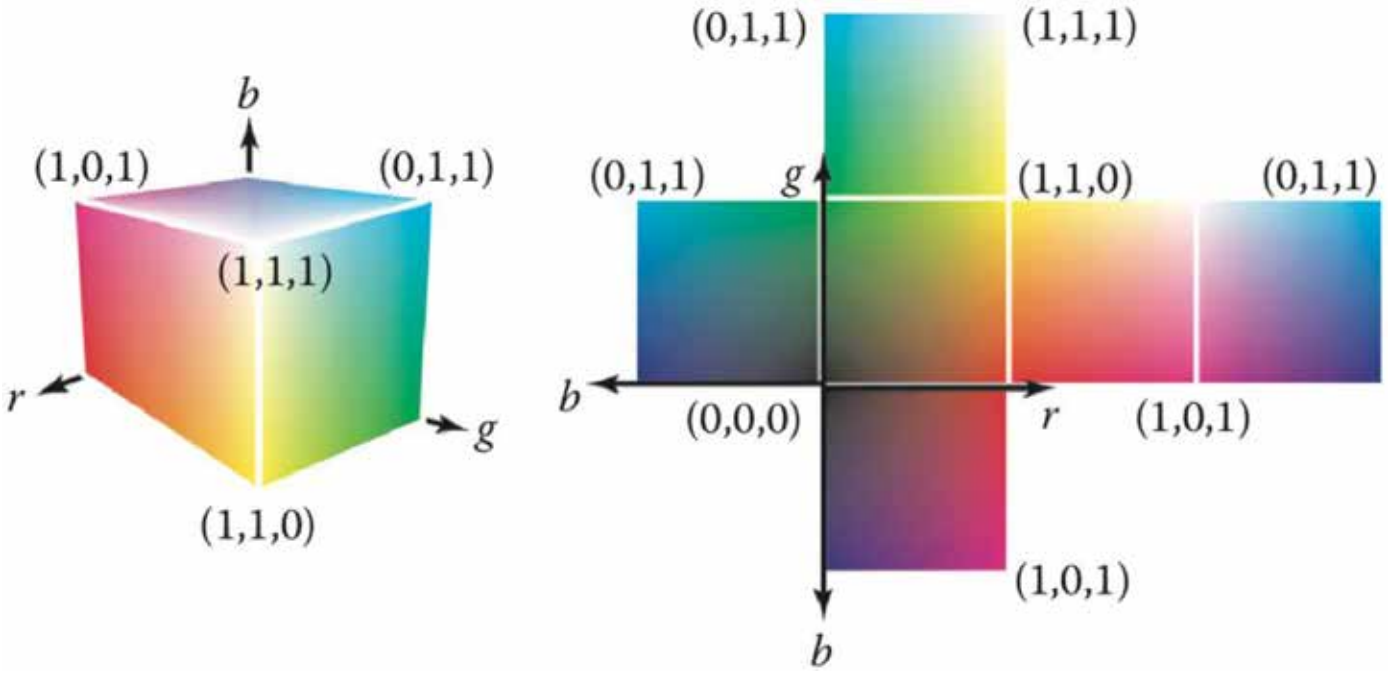


$$i = 0.2126r + 0.7152g + 0.0722b.$$

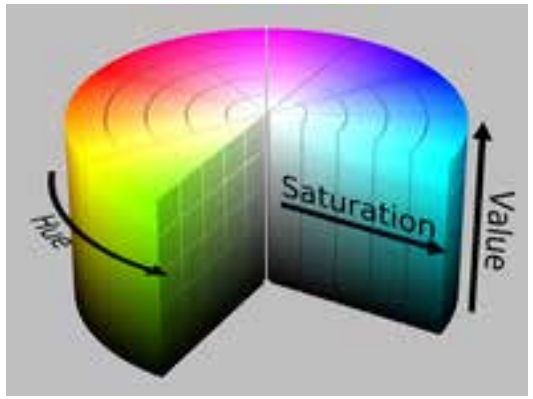
Demosaicing an image



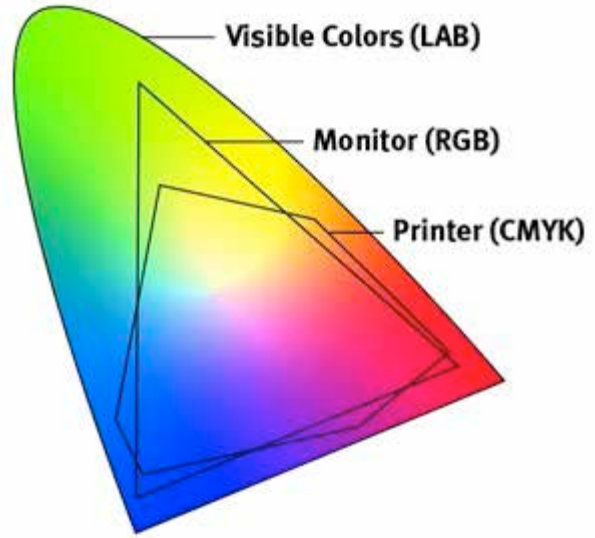
RGB Images



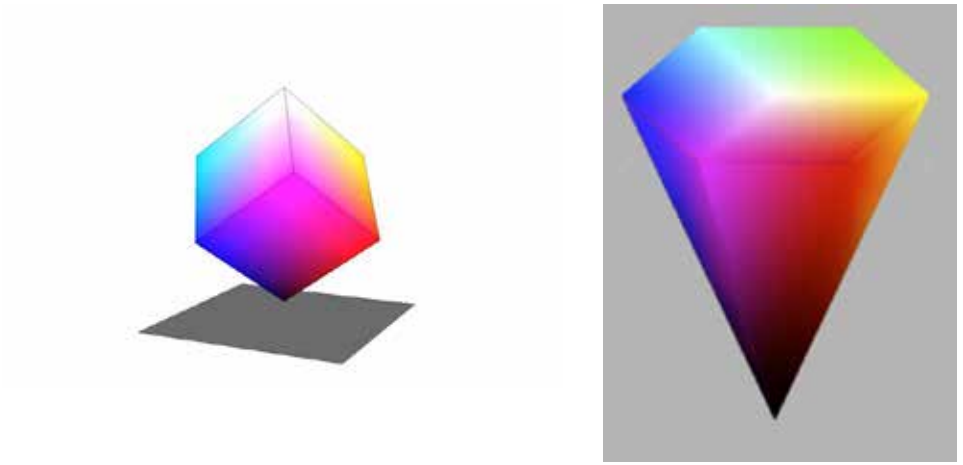
$$\begin{aligned}
\text{black} &= (0, 0, 0), \\
\text{red} &= (1, 0, 0), \\
\text{green} &= (0, 1, 0), \\
\text{blue} &= (0, 0, 1), \\
\text{yellow} &= (1, 1, 0), \\
\text{magenta} &= (1, 0, 1),
\end{aligned}$$

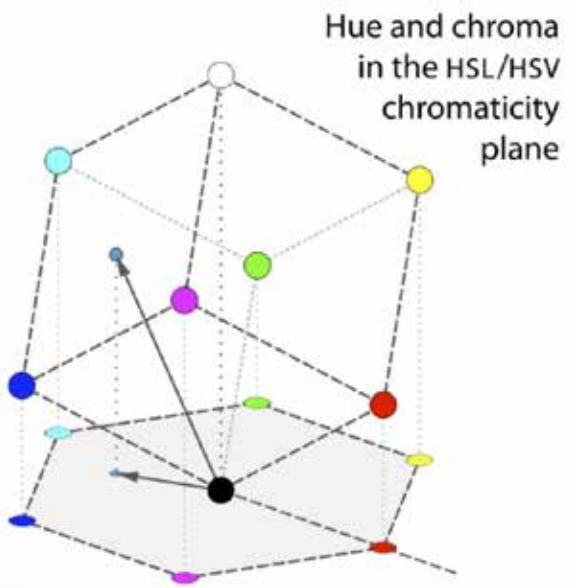


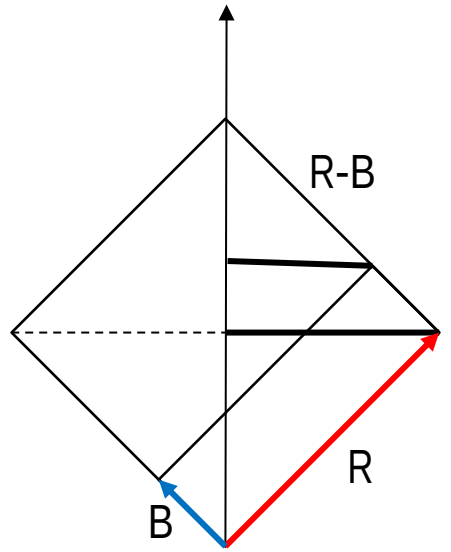
HSV



HSV <=> RGB







V=R

S=(R-B)/R

chroma

$$M = \max(R, G, B)$$
$$m = \min(R, G, B)$$
$$C = \text{range}(R, G, B) = M - m$$

$$V = \max(R, G, B) = M$$

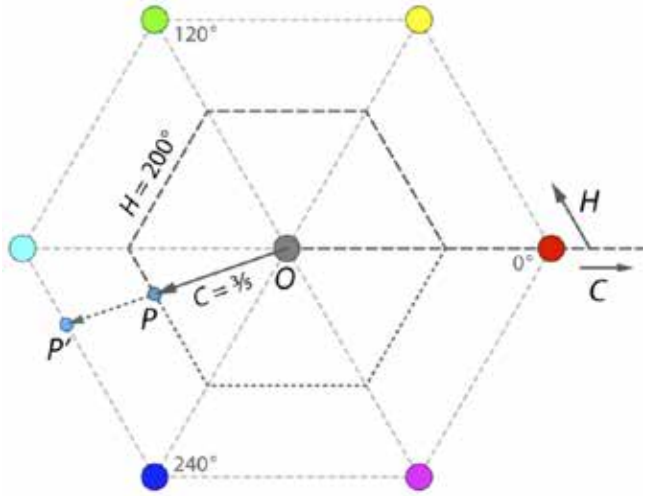
$$S_V = \begin{cases} 0, & \text{if } V = 0 \\ \frac{C}{V}, & \text{otherwise} \end{cases}$$

$$H' = \begin{cases} \text{undefined}, & \text{if } C = 0 \\ \frac{G-B}{C} \bmod 6, & \text{if } M = R \\ \frac{B-R}{C} + 2, & \text{if } M = G \\ \frac{R-G}{C} + 4, & \text{if } M = B \end{cases}$$
$$H = 60^\circ \times H'$$

$R = 1/5$
$G = 3/5$
$B = 4/5$

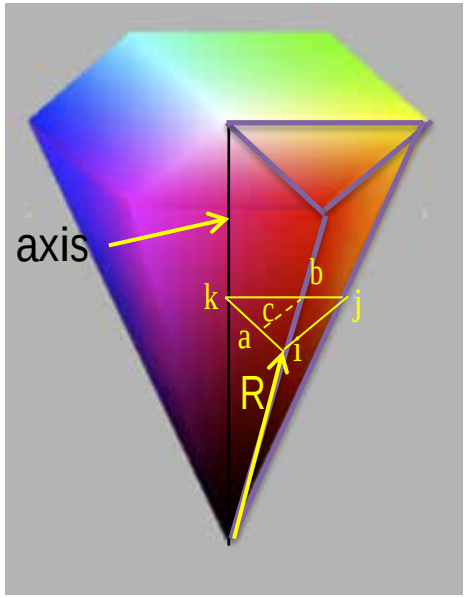
$C = \frac{OP}{OP'} = B - R = 4/5 - 1/5 = 3/5 = .6$

$H = 60° \times \left(4 + \frac{R-G}{C}\right) = 60° \times (4 - 2/3) = 200°$



RGB to HSV

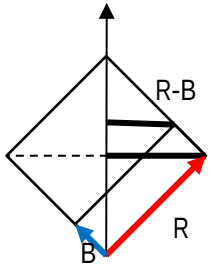
M=max(R,G,B)
m=min(R,G,B)



In the HSV hex-cone:
Value V is the height of the color along the cone axis.
NOTE: The cone axis represents shades of grey i.e. R=G=B, black V=0; white V=1;
All primary colors red, green, blue, cyan, magenta, yellow are hex vertices with V=1,
In general V=M.

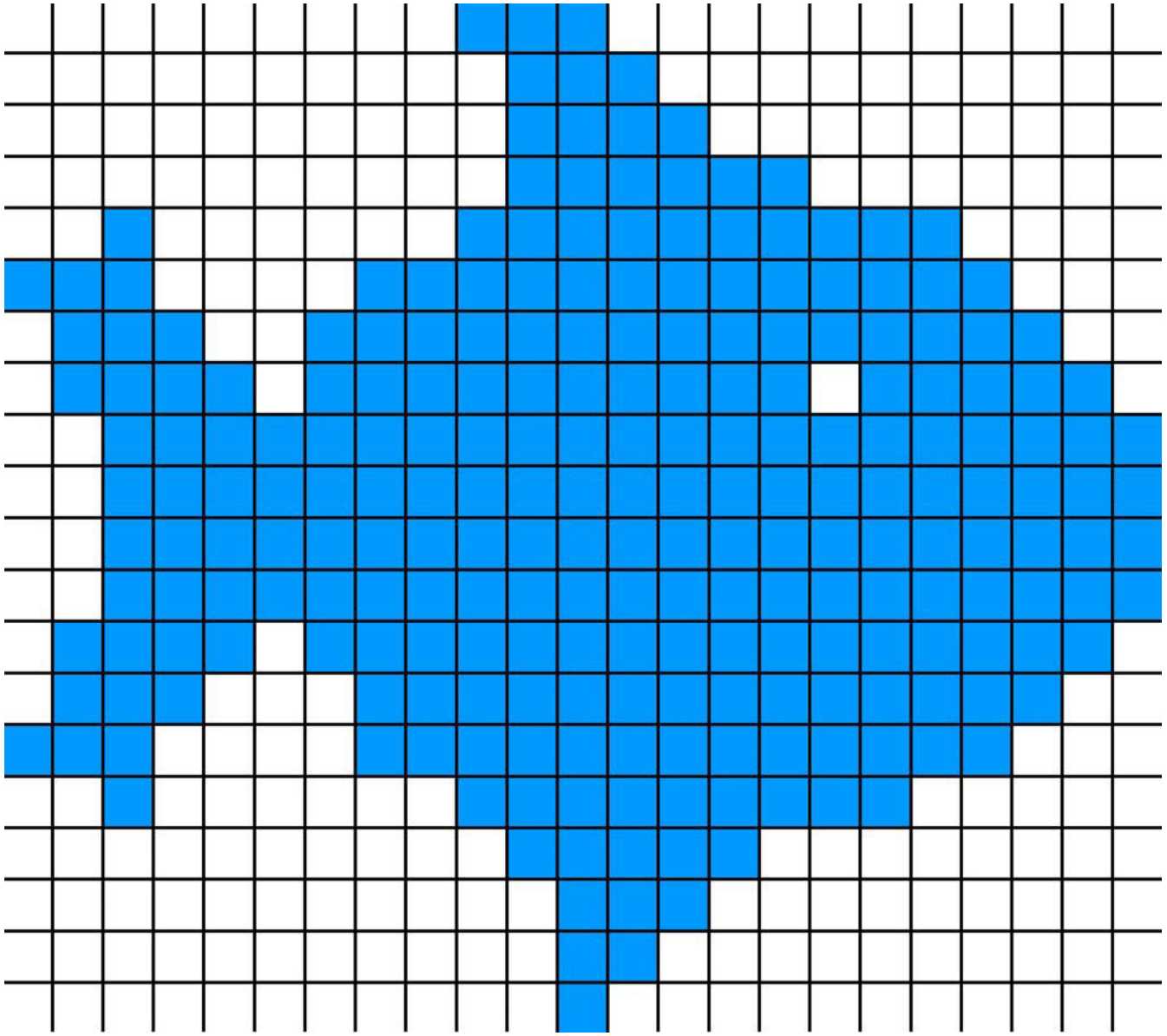
Saturation S=0 along the axis, and S=1 on the surface of the hex-cone.
For a color C=(R,G,B), S=1 iff at least one component is zero, i.e. m=0.
m combined with the other two components adds grey, S is the fraction of color that is not grey
S= (M-m)/M.

Example: assume M=R and m=B, C is in the equilateral triangle (i,j,k) with verts i=(R,0,0), j=(R,R,0) and k=(R,R,R).
C=(R,G,B) lies on the dashed line (a,b) parallel to (i,j), with the length of (a,k)=R-B and S=|a,k|/ |i,k| = (R-B)/R.
Looking at the R,B axis ortho view at right can also provide intuition to why increasing B reduces saturation.



Hue is the angle around the hexagon. H is undefined if S=0. otherwise = |c,a|/|a,b|=(G-B)/(R-B) *60º.
Remember G is a value between B and R (a=(R,B,B) and b=(R,R,B)).
The mod / +2 /+4 places the triangle in the right pie slice (out of 6) so the angle is 0..360º instead of 0..60º.

Raster Image



Data Types for Raster Images

Storage for 1024x1024 image (1 megapixel) = $2^{10} \times 2^{10}$ pixels

bitmap 1 bit per pixel (bpp) : 2^7KB or 128KB (2^3 pixels per byte)

grayscale 8bpp: 1MB

grayscale 16bpp: 2MB

color RGB 24bpp: 3MB

Q: Suppose you have a 767 X 772 rgb image stored in an array called `data`. How would you access the green value at the pixel on the 36th row and 89th column?

A: `data[1 + 3*(88+767*35)]` (Remember C++ starts counting with `0`).

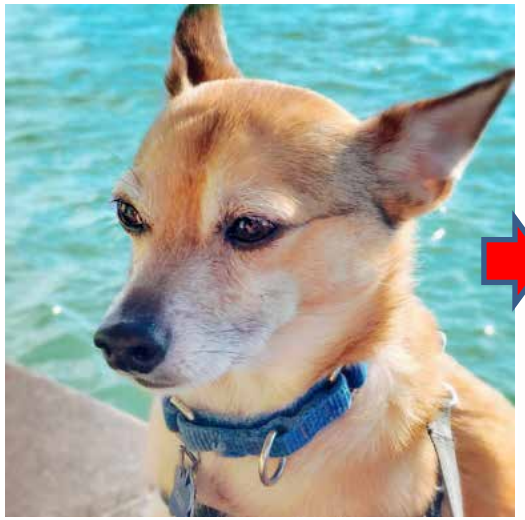
color + alpha (transparency) 32bpp: 4MB

floating-point HDR color: 12MB (each of 3 color channels is a 32bit float i.e. 4B)

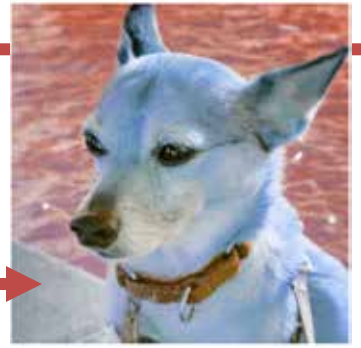
Assignment #1 tasks

File	Marks	What it does
`src/rgba_to_rgb.py`	10	Drop the alpha channel of an RGBA image.
`src/write_ppm.py`	10	Write a grayscale (`P2`) or RGB (`P3`) text `.ppm` file.
`src/reflect.py`	10	Mirror an image horizontally.
`src/rotate.py`	10	Rotate an image 90° counter-clockwise.
`src/rgb_to_gray.py`	10	Perceptual weighted-average grayscale.
`src/rgb_to_hsv.py`	10	Convert RGB → HSV (vectorized).
`src/hsv_to_rgb.py`	10	Convert HSV → RGB (vectorized).
`src/hue_shift.py`	10	Rotate every pixel's hue by a given number of degrees.
`src/desaturate.py`	5	Reduce saturation toward gray by a given factor.
`src/simulate_bayer_mosaic.py`	5	Build a single-channel GBRG mosaic.
`src/demosaic.py`	5	Reconstruct RGB from a Bayer mosaic by interpolation.
`src/over.py`	5	Porter-Duff "over" alpha compositing.

Assignment #1 tasks

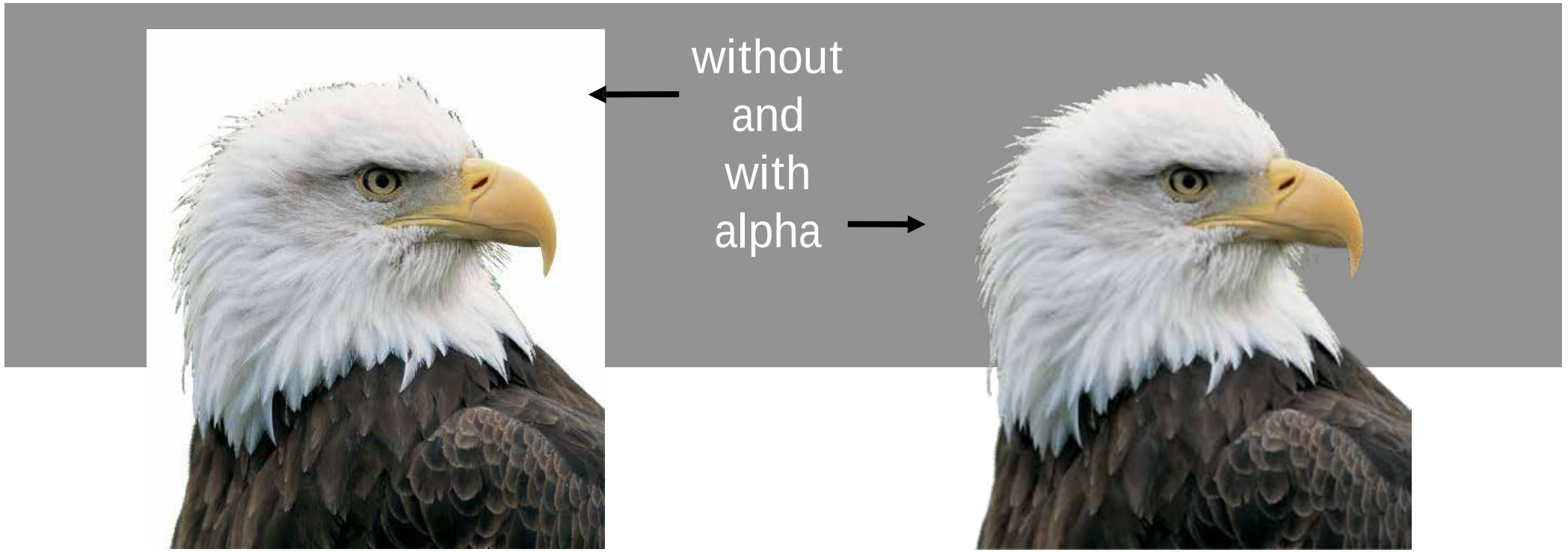


File
src/rgba_to_rgb.py
src/write_ppm.py
src/reflect.py
src/rotate.py
src/rgb_to_gray.py
src/rgb_to_hsv.py
src/hsv_to_rgb.py
src/hue_shift.py
src/desaturate.py
src/simulate_bayer_mosaic.py
src/demosaic.py
src/over.py



Transparency

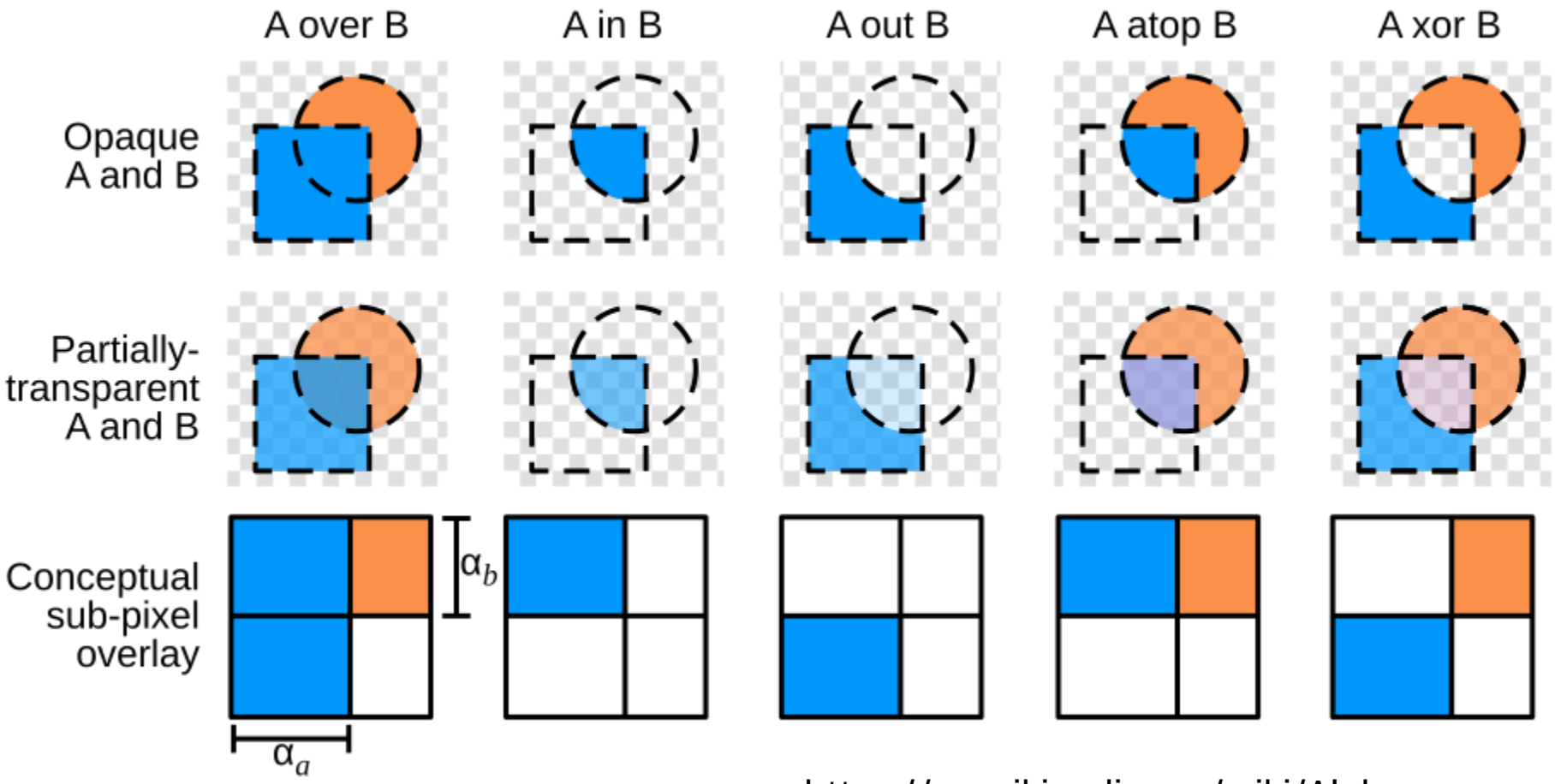
Append (Red, Green, Blue) to be (Red, Green, Blue, Alpha)



$$\mathbf{c} = \alpha \mathbf{c}_f + (1 - \alpha)\mathbf{c}_b.$$

Compositing

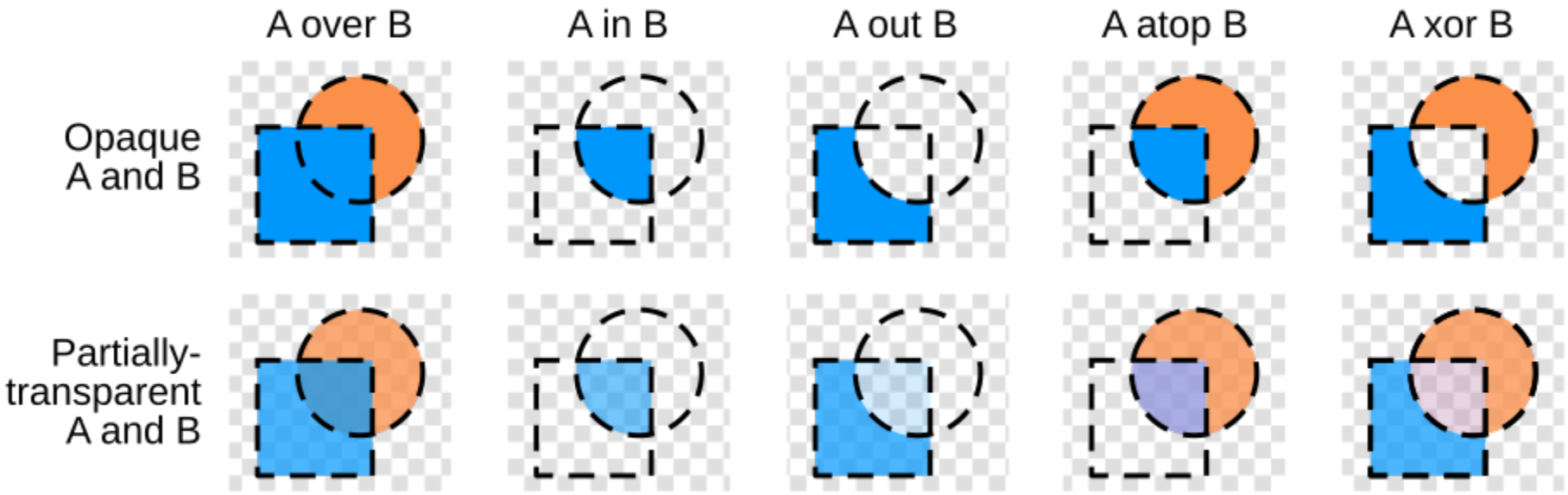
Compositing is about layering images on top of one another



https://en.wikipedia.org/wiki/Alpha_compositing

Compositing

over operator



$$\alpha_o = \alpha_a + \alpha_b(1 - \alpha_a)$$
$$C_o = \frac{C_a\alpha_a + C_b\alpha_b(1 - \alpha_a)}{\alpha_o}$$

$$\alpha_o = \alpha_a(1 - \alpha_b)$$
$$C_o = \frac{C_a\alpha_a(1 - \alpha_b)}{\alpha_o}$$

C_o, C_a and C_b stand for the color components of the pixels in the result image, image A, and image B respectively, applied to each color channel (red/green/blue) individually.
α_o, α_a and α_b are the alpha values of the respective pixels.

Assignment #1 tasks

Compute C = A Over B, where A and B are semi-transparent rgba images and "Over" is the Porter-Duff Over operator.

Raster Image issues: Gamma Correction

Linear encoding V_S =	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
Linear intensity I =	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0

Display intensity is nonlinear wrt input intensity

Raster Image issues: Gamma Correction

$$\text{displayed intensity} = (\text{maximum intensity})a^{\gamma}$$

gamma

... of display

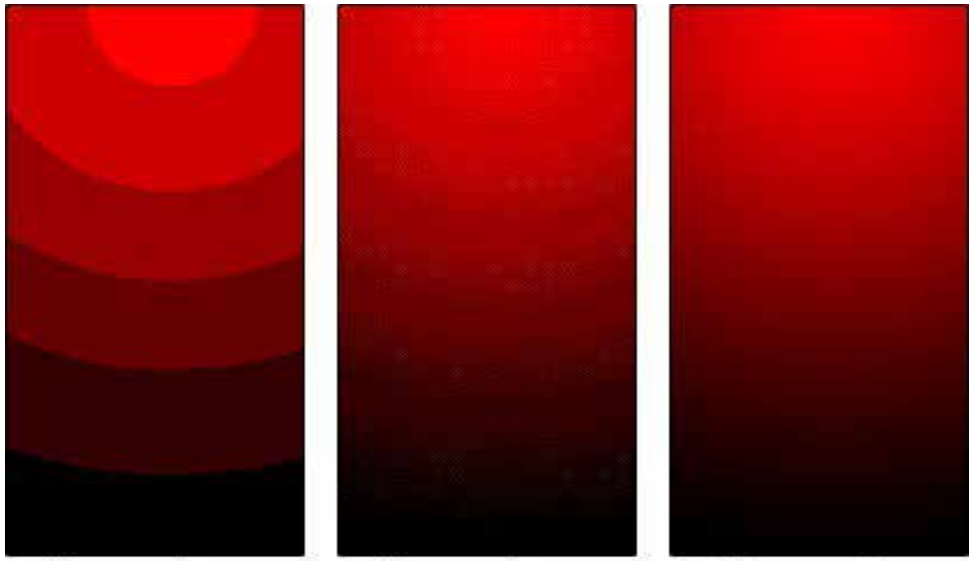
image amplitude [0,1]

Measuring Gamma for a display:

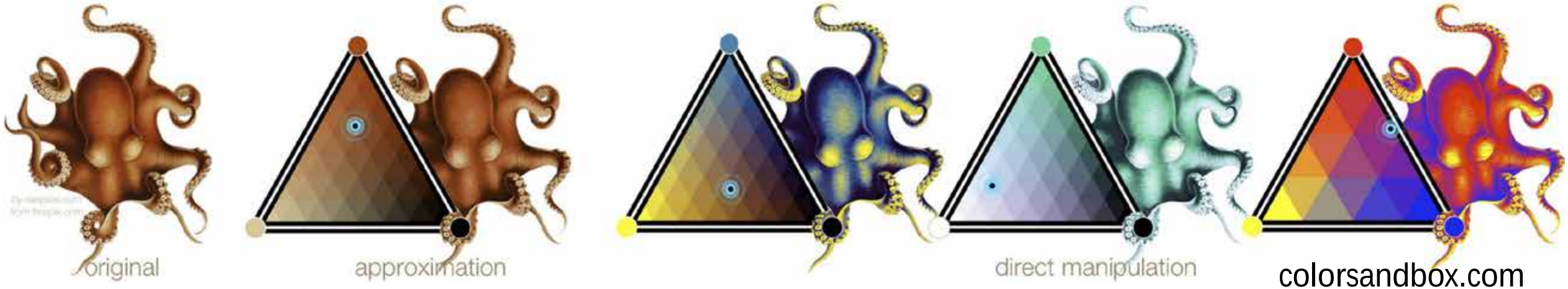
find image amplitude a for ½ display brightness => $\gamma = \frac{\ln 0.5}{\ln a}$

g=0.25 g=0.5 g=1.0 g=1.5 g=2.0

Raster Image issues: Banding

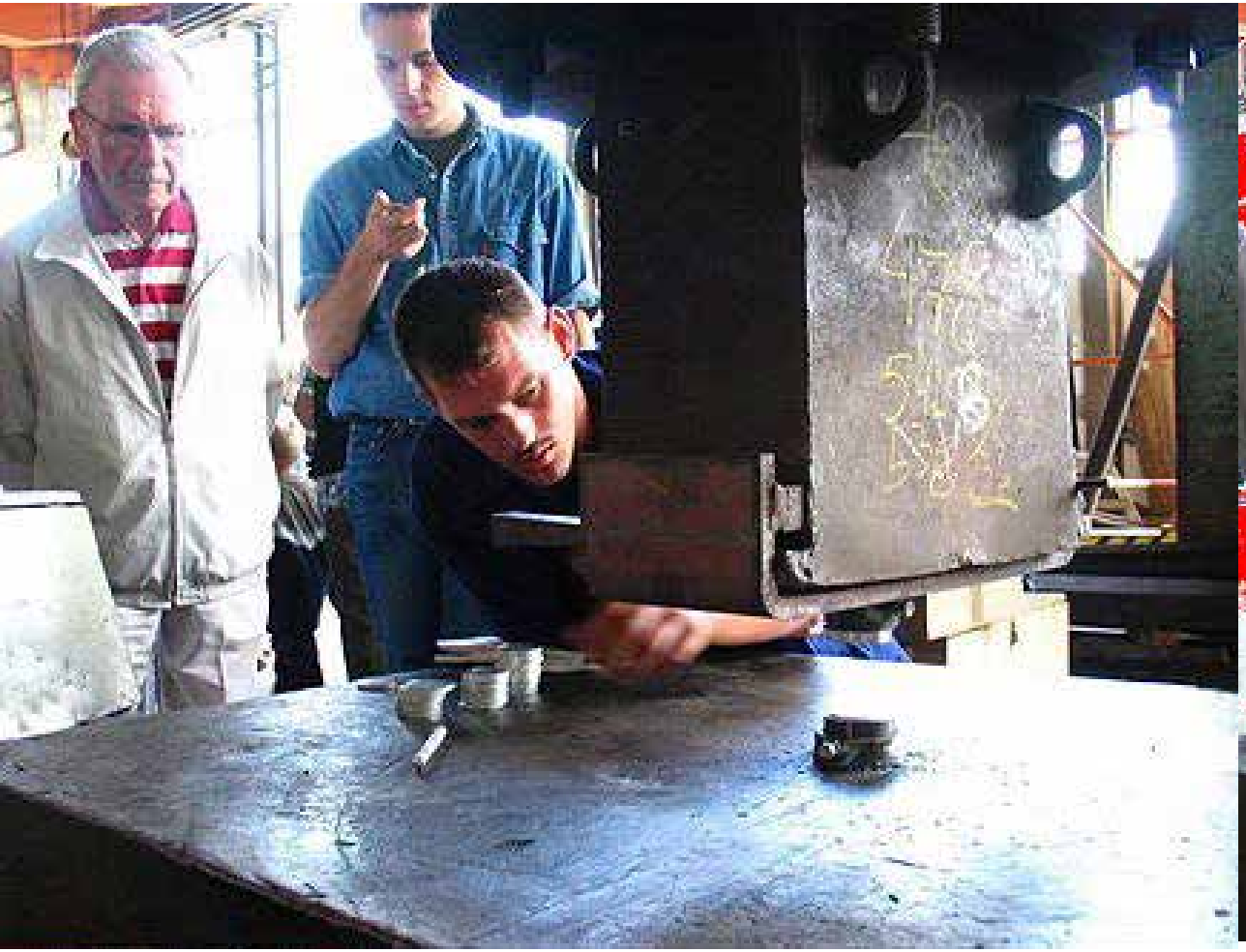


8-bit gradient | 8-bit gradient, dithered | 24-bit gradient



colorsandbox.com

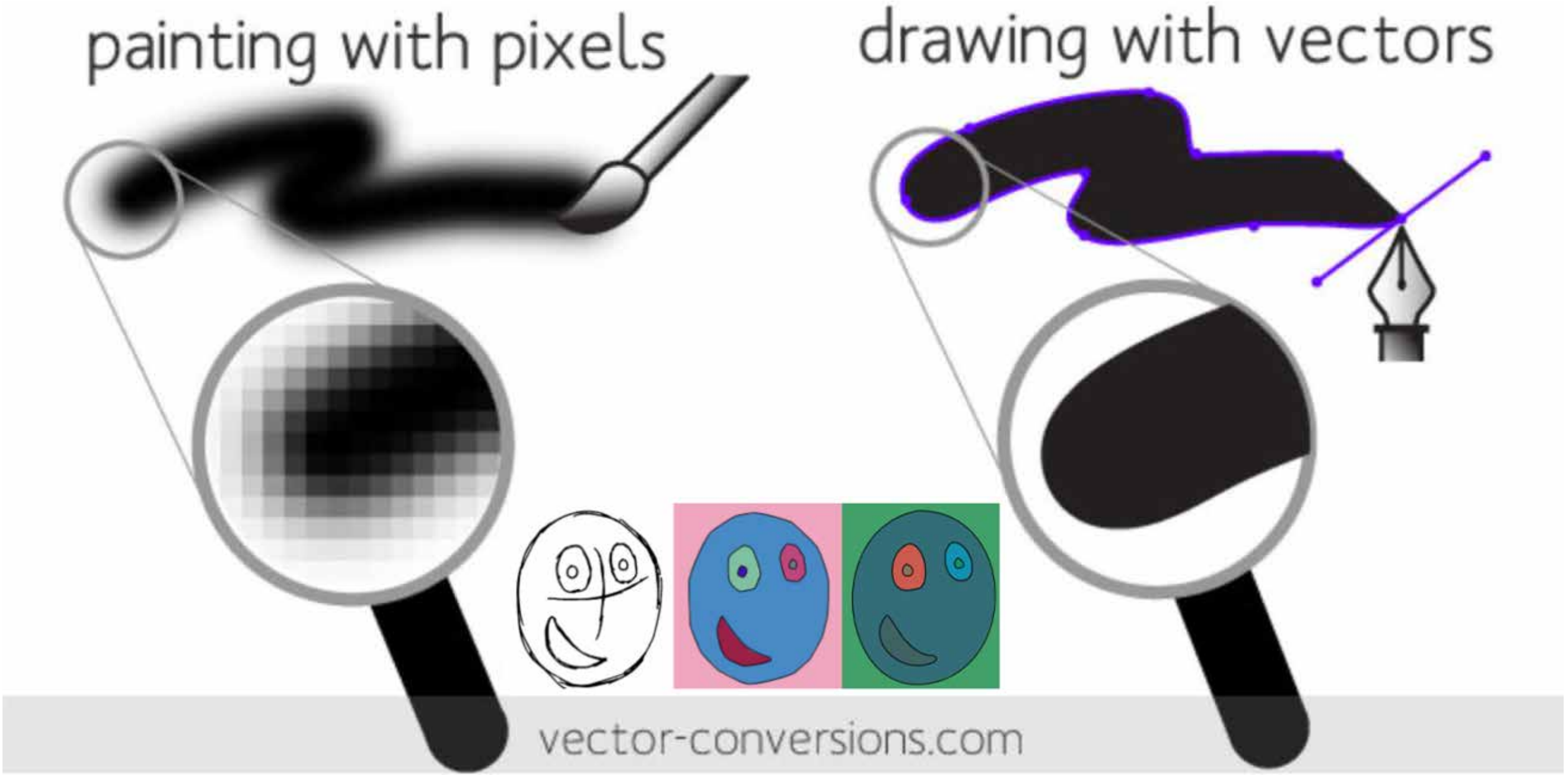
Raster Image issues: Clipping



Original

Clipped

Aside: More Than Just Raster Images



Assignment 1

Available Right Now

Next Week: Ray Casting